ASSIGNMENT -2

1.Serialization:

Serialization is the process of converting a Java object into a byte stream so that it can be stored in a file or transferred over a network.

Java Program

```
import java.io.*;
class Student implements Serializable {
   int id;
   String name;
Student(int id, String name) {
      this.id = id;
      this.name = name;
   }
}
public class SerializationDemo {
   public static void main(String[] args) {
      try {
         Student s = new Student(101, "Sindhu");
         FileOutputStream fos = new FileOutputStream("student.ser");
         ObjectOutputStream oos = new ObjectOutputStream(fos);
         oos.writeObject(s);
         oos.close();
         fos.close();
         System.out.println("Object Serialized Successfully");
      } catch(Exception e) {
         System.out.println(e);
      }
   }
```

}

Output

Object Serialized Successfully

---

2. Exception Handling:

Exception Handling is a mechanism to handle runtime errors without terminating the program.

Java Program

```
public class ExceptionDemo {
   public static void main(String[] args) {
      try {
         int a = 10;
         int b = 0;
         System.out.println(a / b);
      } catch (ArithmeticException e) {
         System.out.println("Cannot divide by zero.");
      }

      System.out.println("Program Continues...");
   }
}
```

Output

Cannot divide by zero.

Program Continues...

---

3. Java Collections:

Collections Framework provides classes to store and manipulate groups of objects.

Java Program

```
import java.util.*;


public class CollectionDemo {
   public static void main(String[] args) {
      ArrayList<String> list = new ArrayList<>();
      list.add("Sindhu");
      list.add("Rahul");
      list.add("Anu");
      for(String name : list){
         System.out.println(name);
      }
   }
}
```

Output

Sindhu

Rahul

Anu

---

4. Priority Queue:

Priority Queue stores elements according to priority.

Java Program

```
import java.util.*;
public class PriorityQueueDemo {
   public static void main(String[] args) {
      PriorityQueue<Integer> pq = new PriorityQueue<>();
      pq.add(50);
      pq.add(20);
      pq.add(10);
```

```
        pq.add(30);
        while(!pq.isEmpty()){
            System.out.println(pq.poll());
        }
    }
}
```

Output

10

20

30

50

---

5. File Handling:

File Handling allows Java programs to create, read and write files.

Java Program

```
import java.io.*;
public class FileDemo {
    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("demo.txt");
            fw.write("Welcome to Java");
            fw.close();
            FileReader fr = new FileReader("demo.txt");
            int ch;
            while((ch = fr.read()) != -1){
                System.out.print((char)ch);
            }
            fr.close();
```

```
    } catch(Exception e){
        System.out.println(e);
    }
  }
}
```

Output

Welcome to Java

---

6. Annotations:

Annotations provide additional information to the compiler or JVM.

Java Program

```
class Parent{
  void display(){
    System.out.println("Parent");
  }
}
class Child extends Parent{
  @Override
  void display(){
    System.out.println("Child");
  }
}
public class AnnotationDemo{
  public static void main(String[] args){
    Child c = new Child();
    c.display();
  }
}
```

Output

Child

---

7. Database Connection (JDBC):

JDBC is an API used to connect Java applications with databases.

Java Program

```
import java.sql.*;
public class JdbcDemo {
   public static void main(String[] args) {
      try {
         Class.forName("com.mysql.cj.jdbc.Driver");
         Connection con = DriverManager.getConnection(
               "jdbc:mysql://localhost:3306/test",
               "root",
               "password");
         System.out.println("Database Connected Successfully");
         con.close();
      } catch(Exception e){
         System.out.println(e);
      }
   }
}
```

Output

Database Connected Successfully

---

8. Design Pattern (Singleton):

Design Patterns are reusable solutions for common software design problems.

Java Program

```
class Singleton {
   private static Singleton obj = new Singleton();
   private Singleton(){}
   public static Singleton getInstance(){
      return obj;
   }

   public void display(){
      System.out.println("Singleton Object Created");
   }
}
public class SingletonDemo {
   public static void main(String[] args){
      Singleton s = Singleton.getInstance();
      s.display();
   }
}
```

Output

Singleton Object Created